

CS 551 Systems Programming

Lecture 3: Expressions and Control Flow

Why the semicolon is load-bearing

Where we are

Lecture 2 gave you enums and `match`, and `match` kept doing something you may not have noticed: **it produced a value.**

That is not a special power of `match`. In Rust almost everything is an **expression**, meaning it evaluates to a value, and the handful of things that are not are exactly where beginners lose an afternoon to a one-character error message.

Today: what an expression is, what a semicolon does to one, and the three loops. By the end we finish the guessing game from Lecture 1 properly, with no `expect` left in it.

Today's path

1. **Expressions and statements.** The semicolon, and the error it causes.
2. **Loops.** `loop`, `while`, `for`, and which to reach for.
3. **Conveniences.** Tuples, `if let`, `let`-else, the turbofish, and the finished guessing game.

3. Expressions and statements

Where the semicolon earns its keep

Two kinds of thing

- A **statement** performs an action and evaluates to nothing.
- An **expression** evaluates to a value.

Rust is expression-oriented: almost everything is an expression. `if` is an expression. `match` is an expression. A block `{ ... }` is an expression.

A semicolon turns an expression into a statement and throws its value away, replacing it with `()`, the empty tuple, called the *unit type*.

That one sentence explains a whole category of compiler errors you are about to meet.

The block value rule

A block evaluates to its **last expression**, if that expression has no semicolon.

```
1  let x = {  
2    let a = 2;  
3    let b = 3;  
4    a + b           // no semicolon: this is the block's value  
5  };  
6  
7  println!("{x}");  // 5
```

Put a semicolon after `a + b` and the block evaluates to `()` instead, and `x` would be `()`.

Functions work the same way

```
1 fn add(a: i32, b: i32) -> i32 {  
2     a + b           // no semicolon: this is the return value  
3 }
```

The return value of a function is the last expression in its body. There is no `return` keyword needed.

`return` does exist, and you use it for **early** returns:

```
1 fn describe(n: i32) -> &'static str {  
2     if n < 0 {  
3         return "negative";  
4     }  
5     "non-negative"  
6 }
```

LIVE CODING CHALLENGE

Write `fn function2(x: i32) -> String` that returns the text `"x = 5"` for an input of `5`. Use `format!`.

One character decides whether it compiles. Which one?

Pause here. Try it before the next slide.

The semicolon that changes the type

```
1 fn function2(x: i32) -> String {  
2     format!("x = {}", x);  
3 }
```

The semicolon that changes the type

```
1 fn function2(x: i32) -> String {  
2     format!("x = {}", x);  
3 }
```

Note the semicolon

The semicolon that changes the type

```
1 fn function2(x: i32) -> String {  
2     format!("x = {}", x);  
3 }
```

The function promises a **String**

What that one character did

```
error[E0308]: mismatched types
|
| fn function2(x: i32) -> String {
|                       ----- expected `String` because of return type
|       format!("x = {}", x);
|                       - help: remove this semicolon to return this value
```

The semicolon turned the expression into a statement and threw its value away, so the body evaluates to `()`.

The compiler tells you exactly what to delete. Read the error; it is more helpful than most languages'.

The semicolon that changes the type

```
1 fn function2(x: i32) -> String {  
2     format!("x = {}", x);  
3 }
```

```
error[E0308]: mismatched types  
|  
| fn function2(x: i32) -> String {  
|           ----- expected `String` because of return type  
|     format!("x = {}", x);  
|           - help: remove this semicolon to return this value
```

The function promises a `String`. The semicolon discarded it, so the body evaluates to `()`. **The compiler tells you exactly what to delete.** Read the error; it is more helpful than most languages'.

`if` is an expression too

```
1 let number = 5;
2
3 let x = if number > 5 {
4     "cool"
5 } else {
6     "not cool"
7 };
8
9 println!("x = {x}");
```

No ternary operator in Rust, because it does not need one. `if` already produces a value.

Note the trailing semicolon after the closing brace. That is the `let` statement ending, not the `if`.

LIVE CODING CHALLENGE

Bind a value with an `if/else`: `"cool"` when `number > 5`, `"not cool"` otherwise. Now put a semicolon after `"cool"`.

What does the compiler say, and why?

Pause here. Try it before the next slide.

The branches must agree

```
1 let number = 5;
2
3 let x = if number > 5 {
4     "cool";
5 } else {
6     "not cool"
7 };
```

```
error[E0308]: `if` and `else` have incompatible types
  |         "cool";
  |         - help: consider removing this semicolon
  | expected `()`, found `&str`
```

One branch has a semicolon, so it evaluates to `()`. The other evaluates to `&str`. An `if` expression has one type, so the two branches have to agree.

CS 551 Same rule as `match` arms. Same rule as function bodies. It is all one rule.

The condition must be a `bool`

```
1 let number = 3;
2
3 if number {
4     println!("nonzero");
5 }
```

```
error[E0308]: mismatched types
|     if number {
|         ^^^^^ expected `bool`, found integer
```

There is no truthiness in Rust. No "zero is false," no "empty string is false," no implicit conversion. If you mean `number != 0`, say so.

4. Loops

loop, while, and for

loop runs until you break out

```
1 let mut n = 0;
2
3 loop {
4     n += 1;
5     if n == 3 {
6         break;
7     }
8 }
9
10 println!("stopped at {n}");
```

loop is an infinite loop. break leaves it.

Why have this when while true exists? Because the compiler knows a loop only exits via break, which lets it reason about what a loop evaluates to. Which brings us to the next slide.

break can carry a value

```
1 let mut attempts = 0;
2
3 let result = loop {
4     attempts += 1;
5     if attempts * attempts > 50 {
6         break attempts;      // this is the value of the loop
7     }
8 };
9
10 println!("{result} attempts"); // 8
```

`loop` is an expression, like everything else. `break value` is how you produce its result.

This is genuinely useful for retry loops: keep trying, and hand back whatever finally worked.

LIVE CODING CHALLENGE

Write a `while` loop that prints every element of `let a = [1, 2, 3, 4, 5];` by index.

Will it compile on the first try?

Pause here. Try it before the next slide.

A `while` loop with a bug

```
1 fn main() {  
2     let a = [1, 2, 3, 4, 5];  
3  
4     let i = 0;  
5  
6     while i < a.len() {  
7         println!("element value: {}", a[i]);  
8         i += 1;  
9     }  
10 }
```

A `while` loop with a bug

```
1 fn main() {  
2     let a = [1, 2, 3, 4, 5];  
3  
4     let i = 0;  
5  
6     while i < a.len() {  
7         println!("element value: {}", a[i]);  
8         i += 1;  
9     }  
10 }
```

But we never declared it **mut**, back on line 4

We are mutating **i** here

A `while` loop with a bug

```
1 fn main() {  
2     let a = [1, 2, 3, 4, 5];  
3  
4     let i = 0;  
5  
6     while i < a.len() {  
7         println!("element value: {}", a[i]);  
8         i += 1;  
9     }  
10 }
```

```
error[E0384]: cannot assign twice to immutable variable `i`
```

`i` is mutated on the last line but never declared `mut`. Bindings are immutable by default, and the compiler holds the line even inside a loop.

for over a collection

```
1 let a = [1, 2, 3, 4, 5];  
2  
3 for element in a.iter() {  
4     println!("element value: {element}");  
5 }
```

for over a collection

```
1 let a = [1, 2, 3, 4, 5];  
2  
3 for element in a.iter() {  
4     println!("element value: {}", element)  
5 }
```

The variable each value from the array gets bound to

for over a collection

```
1 let a = [1, 2, 3, 4, 5];  
2  
3 for element in a.iter() {  
4     println!("element value: {}", element);  
5 }
```

An **iterator**. **for** drives it to exhaustion. Much more on these later

Why this is the loop to reach for

`a.iter()` produces an **iterator**. `for` drives it to exhaustion, binding each item to `element`.

This is the loop you should reach for. Compared to the `while` version:

- No index variable to get wrong.
- No bounds check to write, and no chance of an off-by-one.
- It works on anything iterable, not just things you can index.

for over a range

```
1  for i in 0..5 {  
2      println!("{i}");           // 0 1 2 3 4  
3  }  
4  
5  for i in 1..=5 {  
6      println!("{i}");           // 1 2 3 4 5  
7  }  
8  
9  for i in (1..=5).rev() {  
10     println!("{i}");           // 5 4 3 2 1  
11 }
```

`0..5` is exclusive at the top. `1..=5` is inclusive. A range is a value like any other, and it is iterable.

This is how you loop a fixed number of times. You do not need a mutable counter for it.

5. Conveniences

Tuples, and when a full `match` is too much ceremony

Tuples, when a value is two (or more) values

A tuple groups a fixed number of values, and they do not have to be the same type. You have already met one: `()` is the empty tuple.

```
1 let point = (3, 7);
2 println!("{}", point.0 + point.1); // 10, by position
3
4 let (x, y) = point;                // or destructured, like any other pattern
5 println!("{x} {y}");
```

The type is written the way the value is: `(i32, i32)`. The fields have positions and no names, which is the whole tradeoff. Reach for a tuple when the grouping is small and obvious, and for a `struct`, which Lecture 7 covers, when it is neither.

if let, when you care about one case

A `match` with one interesting arm and a `_` is noisy:

```
1 let config: Option<i32> = Some(7);
2
3 match config {
4     Some(n) => println!("configured: {n}"),
5     _ => {}
6 }
7
8 // same thing, less ceremony
9 if let Some(n) = config {
10     println!("configured: {n}");
11 }
```

`if let` is a `match` with one arm. It is not exhaustive, which is exactly why you use it, and exactly why you should not reach for it when you *should* be handling every case.

The turbofish, when there is no binding to annotate

Lecture 1 picked what `parse` returns by annotating the binding. That annotation does not have to live there. It can go on the call:

```
1 let a: i32 = "41".parse().unwrap();    // on the binding
2 let b = "41".parse::().unwrap();  // on the call
```

`::<>` is the **turbofish**, named for the fish it looks like: `::` the eyes, `<>` the tail. It works on any generic function or method, so `parse::u64()`, `size_of::i32()`, and `collect:: all say the same kind of thing.`

Reach for it when there is no name to annotate. The next slide is exactly that case.

let-else, when the other case gives up

```
1 fn parse_or_zero(s: &str) -> i32 {
2     let Ok(n) = s.parse::<i32>() else {
3         return 0;
4     };
5
6     n           // n is in scope for the rest of the function
7 }
8
9 fn main() {
10     println!("{}", parse_or_zero("41"), parse_or_zero("nope"));
11 }
```

The binding escapes into the surrounding scope, so the happy path stays unindented.

The `else` block **must diverge**: `return`, `break`, `continue`, or `panic!`. It cannot fall through: `n` would be unbound.

if let chains

New in **edition 2024**. Test several patterns in one condition:

```
1 let a: Option<i32> = Some(1);
2 let b: Option<i32> = Some(2);
3
4 if let Some(x) = a
5     && let Some(y) = b
6     && x < y
7 {
8     println!("{x} < {y}");
9 }
```

Before this, that needed nested `if let` blocks or a tuple `match`. Note that this **does not compile under edition 2021**, which is a concrete demonstration of what an edition actually changes.

matches! for a quick yes or no

```
1 let x: Option<i32> = Some(3);
2
3 let has_value = matches!(x, Some(_));
4 let is_small = matches!(x, Some(n) if n < 10);
5
6 println!("{has_value} {is_small}");
```

`matches!` is a macro that returns a `bool`. The `if n < 10` part is a **match guard**, and guards work in ordinary `match` arms too.

Use it when you want a predicate, not a branch.

Constants

```
1 const MAX_GUESSES: u32 = 10;
```

A `const` is not just an immutable `let`:

- **The type annotation is required.** The compiler will not infer it.
- **It can be declared in any scope**, including at the top level of a file.
- **It must be a constant expression**, computable at compile time. It cannot be the result of a normal function call.
- **There is no `mut const`**. The idea does not exist.

```
1 const MAX_GUESSES = 10;
```

```
error: missing type for `const` item
```

Constants

```
1 const MAX_GUESSES: u32 = 10;
```

A `const` keyword an item

The compiler **demand**s a type here, and will not infer one

- **The type annotation is required.** The compiler will not infer it.
- **It can be declared in any scope**, including at the top level of a file.
- **It must be a constant expression**, computable at compile time. It cannot be the result of a normal function call.
- **There is no `mut const`.** The idea does not exist.

```
1 const MAX_GUESSES = 10;
```

```
error: missing type for `const` item
```

LIVE CODING CHALLENGE

Take the Lecture 1 guessing game and finish it: loop until the guess is right, branch with `match` on an `Ordering`, and skip bad input instead of crashing.

Pause here. Try it before the next slide.

The finished guessing game

```
1 use std::cmp::Ordering;
2 use std::io;
3
4 fn main() {
5     let secret_number = rand::random_range(1..=100);
6     loop {
7         println!("Please input your guess.");
8         let mut guess = String::new();
9         io::stdin().read_line(&mut guess).expect("Failed to read line!");
10
11         let Ok(guess) = guess.trim().parse::<i32>() else {
12             println!("That's not a number. Try again.");
13             continue;
14         };
15
16         match guess.cmp(&secret_number) {
17             Ordering::Less => println!("Too small!"),
18             Ordering::Greater => println!("Too big!"),
19             Ordering::Equal => { println!("You got it!"); break; }
20         }
21     }
22 }
```

What changed from Lecture 1

1. `loop` wraps the whole thing, and `break` ends it on a correct guess.
2. `match on Ordering` replaced the `if/else if/else` chain.
3. `let-else with continue` replaced `.expect()` on the parse. Typing `abc` no longer crashes the program; it asks again.

That third change is the one to notice. In Lecture 1, bad input killed the program. Handling it took one pattern and one keyword, and the compiler guaranteed we did not forget.

Aside: where this is going

Rust is experimenting with lifting patterns *into the type system*. It is called **pattern types**:

```
u8 is 1..                // a u8 that is never zero
Option<i32> is Some(_)    // an Option that is never None
&[i32] is [_, ..]        // a slice that is never empty
```

A function could then demand a non-empty slice in its signature and skip the runtime check entirely.

This is `#![feature(pattern_types)]`, nightly only, and the RFC is not approved. Do not use it. It is worth knowing about because it answers a question people ask: no, `match` does not test types today, and this is the proposal that comes closest to blurring that line.

Recap: what Lecture 2 gave you

1. **Enums** name a closed set of possibilities, and variants can carry data. `Option` and `Result` are ordinary enums.
2. **Variants are not types.** `Some` is a value constructor, not a type.
3. `match` is **exhaustive**. Forgetting a case is a compile error, and that is the feature.
4. **Patterns destructure.** Matching pulls values apart and binds the pieces.

Recap: what today added

5. **Almost everything is an expression**, including `if`, `match`, blocks, and `loop`.
6. **A semicolon discards a value** and replaces it with `()`. Most "mismatched types" errors in your first week are this.
7. `for` **over an iterator** is the loop to reach for.
8. `if let`, `let -else`, `matches!` are the low-ceremony forms.
9. **The turbofish** `::<>` puts the type on the call, not on the binding.
10. **A tuple** groups values by position, and `()` is the empty one.

Exercises

1. Add a guess counter to the finished game and print it when the player wins. Use `break` with a value to get the count out of the `loop`.
2. Write `fn sign(n: i32) -> &'static str` returning `"negative"`, `"zero"`, or `"positive"`, using `match` with guards and no `if`.
3. Take the `area` function and add a `Triangle { base: f64, height: f64 }` variant to `Shape`. Do **not** touch `area`. What does the compiler say, and why is that good?

(More) Exercises

4. Predict, for each, whether it compiles. Then check.

- `let x = if true { 1 } else { "two" };`
- `let x = { let a = 1; a };`
- `let x = { let a = 1; a; };` and then `let y: i32 = x;`

5. Rewrite the buggy `while` loop from this lecture as a `for` loop. How many of its bugs become impossible?

6. Write the `if let` chain example using nested `if let` instead. Which reads better?

Reading

- **The Rust Programming Language**, chapters 3 (Common Programming Concepts), 6 (Enums and Pattern Matching), and 18 (Patterns and Matching).
- **Rust by Example**, the `match` and `flow_control` sections.
- **The Edition Guide**, on what edition 2024 changed. You just saw one of them.

Next lecture: memory, pointers, and the borrow checker. The part of Rust that is genuinely new, and the reason the language exists.

Next: Lecture 4

Memory, Pointers, and Box

Where values live, what a pointer carries, and who is allowed to write.